

SHA-2 (Secure Hash Algorithm 2) is a set of cryptographic hash functions designed by the United States National Security Agency (NSA) and first published in 2001. They are built using the Merkle–Damgård construction, from a one-way compression function itself built using the Davies–Meyer structure from a specialized block cipher.

SHA-2 includes significant changes from its predecessor, SHA-1. The SHA-2 family consists of six hash functions with digests (hash values) that are 224, 256, 384 or 512 bits: SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, SHA-512/256. SHA-256 and SHA-512 are hash functions whose digests are eight 32-bit and 64-bit words, respectively. They use different shift amounts and additive constants, but their structures are otherwise virtually identical, differing only in the number of rounds. SHA-224 and SHA-384 are truncated versions of SHA-256 and SHA-512 respectively, computed with different initial values. SHA-512/224 and SHA-512/256 are also truncated versions of SHA-512, but the initial values are generated using the method described in Federal Information Processing Standards (FIPS) PUB 180-4.

SHA-2 was first published by the National Institute of Standards and Technology (NIST) as a U.S. federal standard. The SHA-2 family of algorithms are patented in the U.S. The United States has released the patent under a royalty-free license.

As of 2024, the SHA-2 cryptographic hash function remains secure against known attacks, including those from quantum computers.

Hash standard

With the publication of FIPS PUB 180-2, NIST added three additional hash functions in the SHA family. The algorithms are collectively known as SHA-2, named after their digest lengths (in bits): SHA-256, SHA-384, and SHA-512.

The algorithms were first published in 2001 in the draft FIPS PUB 180-2, at which time public review and comments were accepted. In August 2002, FIPS PUB 180-2 became the new Secure Hash Standard, replacing FIPS PUB 180-1, which was released in April 1995. The updated

standard included the original SHA-1 algorithm, with updated technical notation consistent with that describing the inner workings of the SHA-2 family.

In February 2004, a change notice was published for FIPS PUB 180-2, specifying an additional variant, SHA-224, defined to match the key length of two-key Triple DES. In October 2008, the standard was updated in FIPS PUB 180-3, including SHA-224 from the change notice, but otherwise making no fundamental changes to the standard. The primary motivation for updating the standard was relocating security information about the hash algorithms and recommendations for their use to Special Publications 800-107 and 800-57. Detailed test data and example message digests were also removed from the standard, and provided as separate documents.

In January 2011, NIST published SP800-131A, which specified a move from the then-current minimum of 80-bit security (provided by SHA-1) allowable for federal government use until the end of 2013, to 112-bit security (provided by SHA-2) being both the minimum requirement (starting in 2014) and the recommended security level (starting from the publication date in 2011).

In March 2012, the standard was updated in FIPS PUB 180-4, adding the hash functions SHA-512/224 and SHA-512/256, and describing a method for generating initial values for truncated versions of SHA-512. Additionally, a restriction on padding the input data prior to hash calculation was removed, allowing hash data to be calculated simultaneously with content generation, such as a real-time video or audio feed. Padding the final data block must still occur prior to hash output.

In July 2012, NIST revised SP800-57, which provides guidance for cryptographic key management. The publication disallowed creation of digital signatures with a hash security lower than 112 bits after 2013. The previous revision from 2007 specified the cutoff to be the end of 2010. In August 2012, NIST revised SP800-107 in the same manner.

In March 2023, NIST announced its decision to revise FIPS 180-4. FIPS 180-5 will remove the

SHA-1 specification, add guidance from SP 800-107, and include editorial updates.

The NIST hash function competition selected a new hash function, SHA-3, in 2012. The SHA-3 algorithm is not derived from SHA-2.

Applications

The SHA-2 hash function is implemented in some widely used security applications and protocols, including TLS and SSL, PGP, SSH, S/MIME, and IPsec. The inherent computational demand of SHA-2 algorithms has driven the proposal of more efficient solutions, such as those based on application-specific integrated circuits (ASICs) hardware accelerators.

SHA-256 is used for authenticating Debian software packages and in the DKIM message signing standard; SHA-512 is part of a system to authenticate archival video from the International Criminal Tribunal of the Rwandan genocide. SHA-256 and SHA-512 are used in DNSSEC. Linux distributions usually use 512-bit SHA-2 for secure password hashing.

Several cryptocurrencies, including Bitcoin, use SHA-256 for verifying transactions and calculating proof of work or proof of stake. The rise of ASIC SHA-2 accelerator chips has led to the use of script-based proof-of-work schemes.

In both 4G and 5G mobile networks, HMAC-SHA-256 is utilized as a key derivation function (KDF) to generate cryptographic keys essential for securing communications. This process is defined in the 3rd Generation Partnership Project (3GPP) Technical Specifications TS 33.401 and TS 33.501, which outline the security architecture and procedures for these networks.

SHA-1, SHA-2, and SHA-3 are the Secure Hash Algorithms required by law for use in certain U.S. Government applications, including use within other cryptographic algorithms and protocols, for the protection of sensitive unclassified information. FIPS PUB 180-1 also encouraged adoption and use of SHA-1 by private and commercial organizations. SHA-1 is being retired for most government uses; the U.S. National Institute of Standards and Technology says, "NIST recommends that federal agencies transition away from SHA-1 for all

applications as soon as possible. Federal agencies should use SHA-2 or SHA-3 as an alternative to SHA-1.". NIST's directive that U.S. government agencies ought to, but not explicitly must, stop uses of SHA-1 after 2010 was hoped to accelerate migration away from SHA-1.

The SHA-2 functions were not quickly adopted initially, despite better security than SHA-1. Reasons might include lack of support for SHA-2 on systems running Windows XP SP2 or older and a lack of perceived urgency since SHA-1 collisions had not yet been found. The Google Chrome team announced a plan to make their web browser gradually stop honoring SHA-1-dependent TLS certificates over a period from late 2014 and early 2015. Similarly, Microsoft announced that Internet Explorer and Edge [Legacy] would stop honoring public SHA-1-signed TLS certificates from February 2017. Mozilla disabled SHA-1 in Firefox during early January 2016, but had to re-enable it temporarily via an update, after problems with web-based user interfaces of some router models and security appliances.

Cryptanalysis and validation

For a hash function for which L is the number of bits in the message digest, finding a message that corresponds to a given message digest can always be done using a brute force search in 2^L evaluations. This is called a preimage attack and may or may not be practical depending on L and the particular computing environment. The second criterion, finding two different messages that produce the same message digest, known as a collision, requires on average only $2^{L/2}$ evaluations using a birthday attack.

Some of the applications that use cryptographic hashes, such as password storage, are only minimally affected by a collision attack. Constructing a password that works for a given account requires a preimage attack, as well as access to the hash of the original password (typically in the shadow file) which may or may not be trivial. Reversing password encryption (e.g., to obtain a password to try against a user's account elsewhere) is not made possible by the attacks. (However, even a secure password hash cannot prevent brute-force attacks on

weak passwords.)

In the case of document signing, an attacker could not simply fake a signature from an existing document—the attacker would have to produce a pair of documents, one innocuous and one damaging, and get the private key holder to sign the innocuous document. There are practical circumstances in which this is possible; until the end of 2008, it was possible to create forged SSL certificates using an MD5 collision which would be accepted by widely used web browsers.

Increased interest in cryptographic hash analysis during the SHA-3 competition produced several new attacks on the SHA-2 family, the best of which are given in the table below. Only the collision attacks are of practical complexity; none of the attacks extend to the full round hash function.

At FSE 2012, researchers at Sony gave a presentation suggesting pseudo-collision attacks could be extended to 52 rounds on SHA-256 and 57 rounds on SHA-512 by building upon the biclique pseudo-preimage attack.

Official validation

Implementations of all FIPS-approved security functions can be officially validated through the CMVP program, jointly run by the National Institute of Standards and Technology (NIST) and the Communications Security Establishment (CSE). For informal verification, a package to generate a high number of test vectors is made available for download on the NIST site; the resulting verification, however, does not replace the formal CMVP validation, which is required by law for certain applications.

As of December 2013, there are over 1300 validated implementations of SHA-256 and over 900 of SHA-512, with only 5 of them being capable of handling messages with a length in bits not a multiple of eight while supporting both variants.

Test vectors

Hash values of an empty string (i.e., a zero-length input text).

SHA224("")

0x d14a028c2a3a2bc9476102bb288234c415a2b01f828ea62ac5b3e42f

SHA256("")

0x e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

SHA384("")

0x

38b060a751ac96384cd9327eb1b1e36a21fdb71114be07434c0cc7bf63f6e1da274edebfe76f65fb
d51ad2f14898b95b

SHA512("")

0x

cf83e1357eefb8bdf1542850d66d8007d620e4050b5715dc83f4a921d36ce9ce47d0d13c5d85f2b
0ff8318d2877eec2f63b931bd47417a81a538327af927da3e

SHA512/224("")

0x 6ed0dd02806fa89e25de060c19d3ac86cabb87d6a0ddd05c333b84f4

SHA512/256("")

0x c672b8d1ef56ed28ab87c3622c5114069bdd3ad7b8f9737498d0c01ecef0967a

Even a small change in the message will (with overwhelming probability) result in a different hash, due to the avalanche effect. For example, adding a period to the end of the following sentence changes approximately half (111 out of 224) of the bits in the hash, equivalent to picking a new hash at random:

SHA224("The quick brown fox jumps over the lazy dog")

0x 730e109bd7a8a32b1cb9d9a09aa2325d2430587ddbc0c38bad911525

SHA224("The quick brown fox jumps over the lazy dog.")

0x 619cba8e8e05826e9b8c519c0a5c68f4fb653e8a3d8aa04bb2c8cd4c

Pseudocode

Pseudocode for the SHA-256 algorithm follows. Note the great increase in mixing between bits of the $w[16..63]$ words compared to SHA-1.

Note 1: All variables are 32 bit unsigned integers and addition is calculated modulo 2³²

Note 2: For each round, there is one round constant $k[i]$ and one entry in the message schedule array $w[i]$, $0 \leq i \leq 63$

Note 3: The compression function uses 8 working variables, a through h

Note 4: Big-endian convention is used when expressing the constants in this pseudocode, and when parsing message block data from bytes to words, for example, the first word of the input message "abc" after padding is 0x61626380

Initialize hash values:

(first 32 bits of the fractional parts of the square roots of the first 8 primes 2..19):

$h_0 := 0x6a09e667$

$h_1 := 0xbb67ae85$

$h_2 := 0x3c6ef372$

$h_3 := 0xa54ff53a$

$h_4 := 0x510e527f$

$h_5 := 0x9b05688c$

$h_6 := 0x1f83d9ab$

$h_7 := 0x5be0cd19$

Initialize array of round constants:

(first 32 bits of the fractional parts of the cube roots of the first 64 primes 2..311):

$k[0..63] :=$

0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4,
0xab1c5ed5,
0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7,
0xc19bf174,
0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc,
0x76f988da,
0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351,
0x14292967,
0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e,
0x92722c85,
0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585,
0x106aa070,
0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f,
0x682e6ff3,
0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7,
0xc67178f2

Pre-processing (Padding):

begin with the original message of length L bits

append a single '1' bit

append K '0' bits, where K is the minimum number ≥ 0 such that $(L + 1 + K + 64)$ is a multiple of 512

append L as a 64-bit big-endian integer, making the total post-processed length a multiple of 512 bits

such that the bits in the message are: $\langle \text{original message of length } L \rangle \ 1 \ \langle K \text{ zeros} \rangle \ \langle L \text{ as 64 bit integer} \rangle$, (the number of bits will be a multiple of 512)

Process the message in successive 512-bit chunks:

break message into 512-bit chunks

for each chunk

create a 64-entry message schedule array $w[0..63]$ of 32-bit words

(The initial values in $w[0..63]$ don't matter, so many implementations zero them here)

copy chunk into first 16 words $w[0..15]$ of the message schedule array

Extend the first 16 words into the remaining 48 words $w[16..63]$ of the message schedule array:

for i from 16 to 63

$s_0 := (w[i-15] \text{ rightrotate } 7) \text{ xor } (w[i-15] \text{ rightrotate } 18) \text{ xor } (w[i-15] \text{ rightshift } 3)$

$s_1 := (w[i-2] \text{ rightrotate } 17) \text{ xor } (w[i-2] \text{ rightrotate } 19) \text{ xor } (w[i-2] \text{ rightshift } 10)$

$w[i] := w[i-16] + s_0 + w[i-7] + s_1$

Initialize working variables to current hash value:

$a := h_0$

$b := h_1$

$c := h_2$

$d := h_3$

$e := h_4$

$f := h_5$

$g := h_6$

$h := h_7$

Compression function main loop:

for i from 0 to 63

$S_1 := (e \text{ rightrotate } 6) \text{ xor } (e \text{ rightrotate } 11) \text{ xor } (e \text{ rightrotate } 25)$

$ch := (e \text{ and } f) \text{ xor } ((\text{not } e) \text{ and } g)$

$\text{temp1} := h + S_1 + ch + k[i] + w[i]$

$S_0 := (a \text{ rightrotate } 2) \text{ xor } (a \text{ rightrotate } 13) \text{ xor } (a \text{ rightrotate } 22)$

$\text{maj} := (a \text{ and } b) \text{ xor } (a \text{ and } c) \text{ xor } (b \text{ and } c)$

$\text{temp2} := S_0 + \text{maj}$

$h := g$

$g := f$

$f := e$

$e := d + \text{temp1}$

$d := c$

$c := b$

$b := a$

$a := \text{temp1} + \text{temp2}$

Add the compressed chunk to the current hash value:

$h_0 := h_0 + a$

$h_1 := h_1 + b$

$h_2 := h_2 + c$

$h_3 := h_3 + d$

$h4 := h4 + e$

$h5 := h5 + f$

$h6 := h6 + g$

$h7 := h7 + h$

Produce the final hash value (big-endian):

digest := hash := h0 append h1 append h2 append h3 append h4 append h5 append h6
append h7

The computation of the ch and maj values can be optimized the same way as described for SHA-1.

SHA-224 is identical to SHA-256, except that:

the initial hash values h0 through h7 are different, and

the output is constructed by omitting h7.

SHA-224 initial hash values (in big endian):

(The second 32 bits of the fractional parts of the square roots of the 9th through 16th primes 23..53)

$h[0..7] :=$

0xc1059ed8, 0x367cd507, 0x3070dd17, 0xf70e5939, 0xffc00b31, 0x68581511, 0x64f98fa7,
0xbefa4fa4

SHA-512 is identical in structure to SHA-256, but:

the message is broken into 1024-bit chunks,

the initial hash values and round constants are extended to 64 bits,

there are 80 rounds instead of 64,
the message schedule array w has 80 64-bit words instead of 64 32-bit words,
to extend the message schedule array w, the loop is from 16 to 79 instead of from 16 to 63,
the round constants are based on the first 80 primes 2..409,
the word size used for calculations is 64 bits long,
the appended length of the message (before pre-processing), in bits, is a 128-bit big-endian integer, and
the shift and rotate amounts used are different.

SHA-512 initial hash values (in big-endian):

h[0..7] := 0x6a09e667f3bcc908, 0xbb67ae8584caa73b, 0x3c6ef372fe94f82b,
0xa54ff53a5f1d36f1,
0x510e527fade682d1, 0x9b05688c2b3e6c1f, 0x1f83d9abfb41bd6b,
0x5be0cd19137e2179

SHA-512 round constants:

k[0..79] := 0x428a2f98d728ae22, 0x7137449123ef65cd, 0xb5c0fbcfec4d3b2f,
0xe9b5dba58189dbbc, 0x3956c25bf348b538,
0x59f111f1b605d019, 0x923f82a4af194f9b, 0xab1c5ed5da6d8118,
0xd807aa98a3030242, 0x12835b0145706fbe,
0x243185be4ee4b28c, 0x550c7dc3d5ffb4e2, 0x72be5d74f27b896f,
0x80deb1fe3b1696b1, 0x9bdc06a725c71235,
0xc19bf174cf692694, 0xe49b69c19ef14ad2, 0xefbe4786384f25e3,
0x0fc19dc68b8cd5b5, 0x240ca1cc77ac9c65,
0x2de92c6f592b0275, 0x4a7484aa6ea6e483, 0x5cb0a9dcbd41fbd4,

0x76f988da831153b5, 0x983e5152ee66dfab,
0xa831c66d2db43210, 0xb00327c898fb213f, 0xbf597fc7beef0ee4,
0xc6e00bf33da88fc2, 0xd5a79147930aa725,
0x06ca6351e003826f, 0x142929670a0e6e70, 0x27b70a8546d22ffc,
0x2e1b21385c26c926, 0x4d2c6dfc5ac42aed,
0x53380d139d95b3df, 0x650a73548baf63de, 0x766a0abb3c77b2a8,
0x81c2c92e47edaee6, 0x92722c851482353b,
0xa2bfe8a14cf10364, 0xa81a664bbc423001, 0xc24b8b70d0f89791,
0xc76c51a30654be30, 0xd192e819d6ef5218,
0xd69906245565a910, 0xf40e35855771202a, 0x106aa07032bbd1b8,
0x19a4c116b8d2d0c8, 0x1e376c085141ab53,
0x2748774cdf8eeb99, 0x34b0bcb5e19b48a8, 0x391c0cb3c5c95a63,
0x4ed8aa4ae3418acb, 0x5b9cca4f7763e373,
0x682e6ff3d6b2b8a3, 0x748f82ee5defb2fc, 0x78a5636f43172f60,
0x84c87814a1f0ab72, 0x8cc702081a6439ec,
0x90befffa23631e28, 0xa4506cebd82bde9, 0xbef9a3f7b2c67915,
0xc67178f2e372532b, 0xca273ecee26619c,
0xd186b8c721c0c207, 0xeada7dd6cde0eb1e, 0xf57d4f7fee6ed178,
0x06f067aa72176fba, 0x0a637dc5a2c898a6,
0x113f9804bef90dae, 0x1b710b35131c471b, 0x28db77f523047d84,
0x32caab7b40c72493, 0x3c9ebe0a15c9bebc,
0x431d67c49c100d4c, 0x4cc5d4becb3e42b6, 0x597f299cfc657e2a,
0x5fcb6fab3ad6faec, 0x6c44198c4a475817

SHA-512 Sum & Sigma:

$S0 := (a \text{ rightrotate } 28) \text{ xor } (a \text{ rightrotate } 34) \text{ xor } (a \text{ rightrotate } 39)$

$S1 := (e \text{ rightrotate } 14) \text{ xor } (e \text{ rightrotate } 18) \text{ xor } (e \text{ rightrotate } 41)$

$s0 := (w[i-15] \text{ rightrotate } 1) \text{ xor } (w[i-15] \text{ rightrotate } 8) \text{ xor } (w[i-15] \text{ rightshift } 7)$

$s1 := (w[i-2] \text{ rightrotate } 19) \text{ xor } (w[i-2] \text{ rightrotate } 61) \text{ xor } (w[i-2] \text{ rightshift } 6)$

SHA-384 is identical to SHA-512, except that:

the initial hash values h_0 through h_7 are different (taken from the 9th through 16th primes),
and

the output is constructed by omitting h_6 and h_7 .

SHA-384 initial hash values (in big-endian):

$h[0..7] := 0xcbbb9d5dc1059ed8, \quad 0x629a292a367cd507, \quad 0x9159015a3070dd17,$
 $0x152fec8f70e5939,$
 $0x67332667ffc00b31, \quad 0x8eb44a8768581511, \quad 0xdb0c2e0d64f98fa7,$
 $0x47b5481dbefa4fa4$

SHA-512/t is identical to SHA-512 except that:

the initial hash values h_0 through h_7 are given by the SHA-512/t IV generation function,
the output is constructed by truncating the concatenation of h_0 through h_7 at t bits,
 t equal to 384 is not allowed, instead SHA-384 should be used as specified, and
 t values 224 and 256 are especially mentioned as approved.

SHA-512/224 initial hash values (in big-endian):

$h[0..7] := 0x8c3d37c819544da2, \quad 0x73e1996689dcd4d6, \quad 0x1dfab7ae32ff9c82,$
 $0x679dd514582f9fcf,$

0x0f6d2b697bd44da8, 0x77e36f7304C48942, 0x3f9d85a86a1d36C8,
0x1112e6ad91d692a1

SHA-512/256 initial hash values (in big-endian):

$h[0..7] :=$ 0x22312194fc2bf72c, 0x9f555fa3c84c64c2, 0x2393b86b6f53b151,
0x963877195940eabd,
0x96283ee2a88effe3, 0xbe5e1e2553863992, 0x2b0199fc2c85b8aa,
0x0eb72ddC81c52ca2

The SHA-512/t IV generation function evaluates a modified SHA-512 on the ASCII string "SHA-512/t", substituted with the decimal representation of t. The modified SHA-512 is the same as SHA-512 except its initial values h_0 through h_7 have each been XORed with the hexadecimal constant 0xa5a5a5a5a5a5a5a5.

Sample C implementation for SHA-2 family of hash functions can be found in RFC 6234.

Comparison of SHA functions

In the table below, internal state means the "internal hash sum" after each compression of a data block.

In the bitwise operations column, "Rot" stands for rotate no carry, and "Shr" stands for right logical shift. All of these algorithms employ modular addition in some fashion except for SHA-3.

More detailed performance measurements on modern processor architectures are given in the table below.

The performance numbers labeled 'x86' were running using 32-bit code on 64-bit processors,

whereas the 'x86-64' numbers are native 64-bit code. While SHA-256 is designed for 32-bit calculations, it does benefit from code optimized for 64-bit processors on the x86 architecture. 32-bit implementations of SHA-512 are significantly slower than their 64-bit counterparts. Variants of both algorithms with different output sizes will perform similarly, since the message expansion and compression functions are identical, and only the initial hash values and output sizes are different. The best implementations of MD5 and SHA-1 perform between 4.5 and 6 cycles per byte on modern processors.

Testing was performed by the University of Illinois at Chicago on their hydra8 system running an Intel Xeon E3-1275 V2 at a clock speed of 3.5 GHz, and on their hydra9 system running an AMD A10-5800K APU at a clock speed of 3.8 GHz. The referenced cycles per byte speeds above are the median performance of an algorithm digesting a 4,096 byte message using the SUPERCOP cryptographic benchmarking software. The MiB/s performance is extrapolated from the CPU clockspeed on a single core; real-world performance will vary due to a variety of factors.

Like many other hashes, SHA-2 requires the use of HMAC function to generate a message authentication code (MAC). A naive construction

S

H

A

2

(
k
e
y

|

|

m
e
s
s
a
g
e
)

$$\{\mathrm{SHA2}\}(key \parallel message)$$

cannot be used as a MAC, as it is susceptible to few types of attacks, including the length extension attack.

Implementations

Cryptography libraries that support SHA-2:

Botan

Bouncy Castle

Cryptlib

Crypto++

Libgcrypt

Mbed TLS

libsodium

Nettle

LibreSSL

OpenSSL

GnuTLS

wolfSSL

Hardware acceleration is provided by the following processor extensions:

Intel SHA extensions: Available on some Intel and AMD x86 processors.

VIA PadLock

ARMv8 Cryptography Extensions

IBM z/Architecture: Available since 2005 as part of the Message-Security-Assist Extensions 1 (SHA-256) and 2 (SHA-512)

IBM Power ISA since v.2.07

See also

Comparison of cryptographic hash functions

Comparison of cryptography libraries

Hash function security summary

Hashcash

HMAC

International Association for Cryptologic Research (IACR)

Trusted timestamping

References

Further reading

External links

Descriptions of SHA-256, SHA-384, and SHA-512 from NIST

SHA-2 Checker – SHAChecker to check one's SSL compatibility for SHA-2

SHA-256 Calculator – SHA-256 Calculator

Specifications for a Secure Hash Standard (SHS) – Draft for proposed SHS (SHA-0)

Secure Hash Standard (SHS) – Proposed SHS (SHA-0)

CSRC Cryptographic Toolkit – Official NIST site for the Secure Hash Standard

FIPS PUB 180-4: Secure Hash Standard (SHS) (PDF, 834 KB) – Current version of the Secure Hash Standard (SHA-1, SHA-224, SHA-256, SHA-384, and SHA-512), August 2015

Test vectors for SHA-256/384/512 from the NESSIE project

Test vectors for SHA-1, SHA-2 from NIST site

NIST Cryptographic Hash Project – SHA-3 competition

RFC 3874: "A 224-bit One-way Hash Function: SHA-224"

RFC 6234: "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)"; contains sample C implementation

SHA-256 algorithm demonstration